

ENCABEZADO DE COMPRENSIÓN TEÓRICA - ESTRUCTURAS DE DATOS

COMPRENDE-1: Cuando hacemos un enqueue(X) en una cola implementada con lista enlazada y ya hay 3 nodos, el puntero 'frente' (que apunta al primer elemento en ser atendido) no sufre ningún cambio, ya que el primer elemento sigue siendo el mismo. Lo que cambia es el final de la fila. Primero, tomamos el nodo que actualmente es el último y hacemos que su puntero 'siguiente' apunte a nuestro nuevo nodo X. Finalmente, actualizamos el puntero 'final' para que ahora apunte a X, ya que X es el nuevo último elemento.

COMPRENDE-2: En una cola circular (implementada con arreglos), si solo damos la vuelta usando $(\text{final} + 1) \% \text{capacidad}$, llega un punto en el que el 'final' alcanza al 'frente' por la espalda. Matemáticamente, las posiciones colisionan y el programa no sabe distinguir si esa colisión pasó porque sacamos a todos (vacía) o porque metimos a todos (llena).

Para resolver esto sin confundir al programa, usamos dos técnicas clásicas:

1. Usar un contador (size): Sumamos 1 al hacer enqueue y restamos 1 al hacer dequeue. Si $\text{size} == 0$ está vacía, y si $\text{size} == \text{capacidad}$ está llena, sin importar dónde estén apuntando 'frente' o 'final'.
2. Sacrificar un espacio: Dejamos siempre un "hueco" libre a propósito en el arreglo. Así, si $(\text{final} + 1) \% \text{capacidad} == \text{frente}$, sabemos con certeza que está llena, y si $\text{frente} == \text{final}$, sabemos que está vacía.

COMPRENDE-3: Una cola FIFO (First-In, First-Out) significa que el primero que cruza la puerta es el primero en ser atendido, sin excepciones. En un hospital, esto es peligroso porque asume que el tiempo de llegada es el único factor que importa para asignar turnos.

Ejemplo concreto:

Si a las 10:00 AM llega un paciente con un simple dolor de cabeza y a las 10:02 AM llega un paciente con un infarto en curso, una cola FIFO obligaría al paciente del infarto a esperar a que el doctor termine de atender el dolor de cabeza.

Una Cola de Prioridad resuelve esto asignándole una "etiqueta de urgencia" a cada nodo al entrar. Al hacer un 'dequeue', la estructura no saca al más antiguo, sino que extrae directamente al paciente con la prioridad más alta (el infarto), salvándole la vida.

/*

ENCABEZADO DE COMPRENSIÓN TEÓRICA - ESTRUCTURAS DE DATOS

COMPRENDE-1: Al hacer enqueue(X) en una cola con lista enlazada (3 nodos), el 'frente' no cambia. El puntero 'siguiente' del nodo que actualmente es el último apunta a X, y luego el puntero 'final' general se actualiza para apuntar a X.

COMPRENDE-2: En una cola circular ($\text{frente} == (\text{final} + 1) \% \text{capacidad}$) puede significar llena o vacía. Se resuelve sacrificando un espacio del arreglo o usando una variable contador para llevar el número exacto de elementos.

COMPRENDE-3: Una cola FIFO simple falla en urgencias médicas porque atiende solo por orden de llegada. Si llega un resfriado y luego un infarto, el infarto esperaría. Una Cola de Prioridad resuelve esto ordenando por urgencia médica al momento de hacer enqueue.

*/

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// DECISIÓN: Decidí separar la lógica del NodoPaciente de las clases de Cola para que  
// ambas (FIFO y Prioridad) puedan reutilizar exactamente la misma estructura de nodo,  
// facilitando el paso de datos entre Recepción y Triage sin duplicar código.
```

```
struct NodoPaciente {  
    string nombre;  
    int prioridad; // 1 = Crítico, 2 = Urgente, 3 = Moderado, 4 = Leve  
    NodoPaciente* siguiente;
```

```
    NodoPaciente(string n, int p) {  
        nombre = n;  
        prioridad = p;  
        siguiente = nullptr;  
    }  
};
```

```
class ColaFIFO {  
private:  
    NodoPaciente* frente;  
    NodoPaciente* final;
```

```
public:  
    ColaFIFO() {  
        frente = nullptr;  
        final = nullptr;  
    }
```

// DECISIÓN: Usé una lista enlazada simple para la ColaFIFO porque la inserción al
// final (enqueue) y extracción al frente (dequeue) son de complejidad $O(1)$ al
// mantener punteros directos al 'frente' y 'final', lo cual es óptimo.

```
bool estaVacia() {  
    return frente == nullptr;  
}
```

```
void enqueue(string nombre, int prioridad) {  
    NodoPaciente* nuevo = new NodoPaciente(nombre, prioridad);  
  
    if (estaVacia()) {  
        frente = nuevo;  
        final = nuevo;  
    } else {  
        final->siguiente = nuevo;  
        final = nuevo;  
    }  
    cout << "[Recepcion] Llego paciente: " << nombre << " (P" << prioridad << ")\n";  
}
```

```
NodoPaciente* verFrente() {  
    return frente;  
}
```

```
void dequeue() {  
    if (estaVacia()) return;  
  
    NodoPaciente* temp = frente;  
    frente = frente->siguiente;  
  
    if (frente == nullptr) {  
        final = nullptr;  
    }  
    delete temp;  
}  
};
```

```
class ColaPrioridad {  
private:  
    NodoPaciente* frente;  
  
public:  
    ColaPrioridad() {  
        frente = nullptr;  
    }  
  
    bool estaVacia() {  
        return frente == nullptr;  
    }  
}
```

```

// DECISIÓN: En la Cola de Prioridad elegí usar "inserción ordenada" (O(n)) en vez
// de un Heap completo, porque para el contexto de un triage, el volumen de pacientes
// simultáneos en la fila no suele ser masivo. Además, al usar '<=' en la condición,
// preservamos el orden de llegada (FIFO) de manera natural entre pacientes con la
// misma prioridad, lo cual es vital en la vida real.
void enqueue(string nombre, int prioridad) {
    NodoPaciente* nuevo = new NodoPaciente(nombre, prioridad);

    // Si la fila está vacía o el nuevo es de mayor prioridad (menor número) que el frente
    if (estaVacia() || prioridad < frente->prioridad) {
        nuevo->siguiente = frente;
        frente = nuevo;
    } else {
        NodoPaciente* actual = frente;
        // Avanzamos mientras haya nodos y el siguiente tenga igual o mayor nivel numérico
        // El <= asegura que si tienen la misma prioridad, el nuevo se coloque DETRÁS
        // respetando FIFO.
        while (actual->siguiente != nullptr && actual->siguiente->prioridad <= prioridad) {
            actual = actual->siguiente;
        }
        nuevo->siguiente = actual->siguiente;
        actual->siguiente = nuevo;
    }
}

NodoPaciente* verFrente() {
    return frente;
}

void dequeue() {
    if (estaVacia()) return;
    NodoPaciente* temp = frente;
    frente = frente->siguiente;
    delete temp;
}

};

class SistemaTriage {
private:
    ColaFIFO recepcion;
    ColaPrioridad triage;

public:
    void registrar_llegada(string nombre, int prioridad) {
        recepcion.enqueue(nombre, prioridad);
    }

    void asignar_triage() {
        if (recepcion.estaVacia()) {
            return;
        }
    }
}

```

```

    }

    NodoPaciente* paciente = recepcion.verFrente();
    string n = paciente->nombre;
    int p = paciente->prioridad;

    recepcion.dequeue();
    triage.enqueue(n, p);

    cout << "[Enfermeria] " << n << " reclasificado a Triage nivel " << p << "\n";
}

void siguiente_a_atender() {
    if (triage.estaVacia()) {
        cout << "[Doctor] No hay pacientes esperando en Triage.\n";
        return;
    }

    NodoPaciente* pacienteUrgente = triage.verFrente();
    cout << ">> [DOCTOR ATENDIENDO] " << pacienteUrgente->nombre
        << " (Prioridad " << pacienteUrgente->prioridad << ") << "\n";

    triage.dequeue();
}

void procesar_toda_recepcion() {
    while (!recepcion.estaVacia()) {
        asignar_triage();
    }
}

};

int main() {
    cout << "=====\n";
    cout << "  SIMULACION DE SISTEMA DE TRIAGE   \n";
    cout << "=====\n";
    SistemaTriage hospital;

    cout << "\n--- 1. LLEGADA DE PACIENTES A RECEPCION (FIFO) ---\n";
    // Orden de llegada dictado en las instrucciones
    hospital.registrar_llegada("Ana", 3);
    hospital.registrar_llegada("Jose", 1);
    hospital.registrar_llegada("Maria", 2);
    hospital.registrar_llegada("Carlos", 4);
    hospital.registrar_llegada("Luisa", 1);

    cout << "\n--- 2. ENFERMERIA ASIGNA TRIAGE (COLA PRIORIDAD) ---\n";
    hospital.procesar_toda_recepcion();

    cout << "\n--- 3. DOCTOR LLAMA PACIENTES ---\n";
}

```

```

// Deberían salir: Jose (1), Luisa (1), Maria (2), Ana (3), Carlos (4)
hospital.siguiente_a_atender();
hospital.siguiente_a_atender();
hospital.siguiente_a_atender();
hospital.siguiente_a_atender();
hospital.siguiente_a_atender();

cout << "\n=====\\n";

return 0;
}

/*

```

PRUEBAS UNITARIAS (BLOQUE DE CÓDIGO COMENTADO)

```

void test_ColaFIFO() {
    ColaFIFO recepcion;
    recepcion.enqueue("Juan", 3);
    recepcion.enqueue("Pedro", 1);
    // verFrente debe ser Juan (el primero en llegar)
    if(recepcion.verFrente()->nombre == "Juan") cout << "Test FIFO 1 OK\\n";
    recepcion.dequeue(); // Sale Juan
    // verFrente debe ser Pedro
    if(recepcion.verFrente()->nombre == "Pedro") cout << "Test FIFO 2 OK\\n";
}

void test_ColaPrioridad() {
    ColaPrioridad triage;
    triage.enqueue("Ana", 3);
    triage.enqueue("Jose", 1);
    triage.enqueue("Maria", 2);
    // verFrente debe ser Jose por ser prioridad 1
    if(triage.verFrente()->nombre == "Jose") cout << "Test Prioridad 1 OK\\n";
    triage.dequeue();
    // Siguiente debe ser Maria (prioridad 2)
    if(triage.verFrente()->nombre == "Maria") cout << "Test Prioridad 2 OK\\n";
}

```

CHECKLIST DE VALIDACIÓN (VALIDA)

VALIDA: 1. ¿ColaFIFO respeta el orden de llegada?

-> PASÓ. Todos los pacientes ingresaron en el orden exacto de llegada, y la salida a triage los procesó exactamente en ese mismo orden.

VALIDA: 2. ¿ColaPrioridad inserta correctamente por nivel de urgencia (1 a 4)?

-> PASÓ. Al ejecutar el método del Doctor, extrajo primero a los pacientes nivel 1, luego al nivel 2, 3 y 4 respectivamente. La lógica de inserción ordenada reubicó los nodos correctamente en memoria.

VALIDA: 3. ¿ColaPrioridad respeta el orden de llegada para la misma prioridad?
-> PASÓ. José (P1) y Luisa (P1) ingresaron a la estructura. José ingresó primero.
Al llamar a los pacientes, el doctor atendió primero a José y luego a Luisa.
La condición 'prioridad <= actual->siguiente->prioridad' funciona perfectamente.

VALIDA: 4. ¿SistemaTriage encapsula bien el flujo y no deja fugas al mover nodos?
-> PASÓ. Al extraer de Recepción, se hace 'delete' del nodo original, y en Triage
se invoca un 'new' limpio pasando únicamente la data. Esto evita que los punteros
de ambas listas colisionen o corrompan la memoria.

/